DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Assignment : 1

Linear Search and Binary Search

Name of Student	
Roll Number	
Course/Class	
Date of Experiment	
Faculty Name	

1 Experiment Title

Implementation and analysis of Linear Search and Binary Search using the C programming language.

2 Aim

To study, implement, and compare Linear Search and Binary Search algorithms using C language.

3 Learning Objectives

After completing this experiment, students will be able to:

- Understand the concept of searching.
- Implement Linear Search using C.
- Implement Binary Search using C.
- Understand the requirement of a sorted array for Binary Search.
- Calculate the number of comparisons.
- Analyse the time and space complexities of both algorithms.

4 Software Requirements

- C compiler such as GCC, Code::Blocks, Dev-C++, or Turbo C.
- Operating system: Windows, Linux, or macOS.

5 Linear Search

5.1 Definition

Linear Search

Linear Search is a searching technique in which the elements of an array are checked sequentially, starting from the first element, until the required element is found or the end of the array is reached.

Linear Search is also called **Sequential Search**. It can be applied to both sorted and unsorted arrays.

5.2 Characteristics of Linear Search

1. It checks the array elements one by one.
2. It can be used on sorted and unsorted arrays.
3. It is simple to understand and implement.
4. It does not require preprocessing or sorting.
5. It is suitable for small datasets.
6. Its worst-case time complexity is $O(n)$.
7. It returns the index of the element when the element is found.
8. It returns -1 when the element is not present.

5.3 Algorithm for Linear Search

Let A be an array containing n elements and let `key` be the element to be searched.

1. Start from the first element, that is, index 0.
2. Compare $A[i]$ with `key`.
3. If $A[i] = key$, return index i.
4. Otherwise, move to the next element.
5. Repeat the process until all elements are checked.
6. If the key is not found, return -1.

5.4 Pseudocode for Linear Search

```
LINEAR_SEARCH(A, n, key)

1. for i = 0 to n - 1
2.     if A[i] == key
3.         return i
4.     end if
5. end for
6. return -1
```

5.5 C Program for Linear Search

```c
#include <stdio.h>

int linearSearch(int array[], int size, int key)
{
    int i;

    for (i = 0; i < size; i++)
    {
        if (array[i] == key)
        {
            return i;
        }
    }

    return -1;
}

int main()
{
    int array[100];
    int size, key, result;
    int i;

    printf("Enter the number of elements: ");
    scanf("%d", &size);

    printf("Enter %d elements:\n", size);

    for (i = 0; i < size; i++)
    {
        scanf("%d", &array[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &key);

    result = linearSearch(array, size, key);
```

```
38
39      if (result == -1)
40      {
41          printf("Element not found.\n");
42      }
43      else
44      {
45          printf("Element found at index %d.\n", result);
46          printf("Element found at position %d.\n", result + 1);
47      }
48
49      return 0;
50  }
```

5.6 Sample Input and Output

```
Enter the number of elements: 6
Enter 6 elements:
10 25 30 45 60 75
Enter the element to search: 45

Element found at index 3.
Element found at position 4.
```

5.7 Complexity of Linear Search

Case	Complexity	Condition
Best Case	$O(1)$	The key is present at the first index.
Average Case	$O(n)$	The key is present somewhere in the middle.
Worst Case	$O(n)$	The key is at the last index or is not present.
Space Complexity	$O(1)$	Only a constant amount of additional memory is required.

Table 1: Complexity of Linear Search

6 Binary Search

6.1 Definition

Binary Search

Binary Search is a searching technique that repeatedly divides a sorted array into two halves and selects the half in which the required element may be present.

Binary Search can only be directly applied to an array arranged in **sorted order**.

6.2 Characteristics of Binary Search

1. The array must be sorted.
2. It follows the Divide-and-Conquer technique.
3. It compares the key with the middle element.
4. Half of the remaining elements are eliminated after every comparison.
5. It is faster than Linear Search for large sorted arrays.
6. It can be implemented iteratively or recursively.
7. Its worst-case time complexity is $O(\log n)$.
8. It is efficient when repeated searching is required.

6.3 Middle Index Formula

The basic formula for calculating the middle index is:

$$\boxed{mid = \left\lfloor \frac{low + high}{2} \right\rfloor}$$

In C language, the formula is written as:

```
mid = (low + high) / 2;
```

Since `low`, `high`, and `mid` are integer variables, C automatically removes the decimal part.

For example, if:

$$low = 5 \qquad \text{and} \qquad high = 8,$$

then:

$$mid = \left\lfloor \frac{5 + 8}{2} \right\rfloor = \lfloor 6.5 \rfloor = 6.$$

6.4 Algorithm for Binary Search

Let A be a sorted array containing n elements.

1. Set `low` to 0.
2. Set `high` to $n - 1$.
3. Calculate:
$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

4. If $A[mid] = key$, return `mid`.
5. If $key < A[mid]$, set:
$$high = mid - 1$$
6. If $key > A[mid]$, set:
$$low = mid + 1$$
7. Repeat the steps while:
$$low \leq high$$
8. If the key is not found, return -1.

6.5 Pseudocode for Binary Search

```
BINARY_SEARCH(A, n, key)

1. low = 0
2. high = n - 1

3. while low <= high
4.     mid = floor((low + high) / 2)

5.     if A[mid] == key
6.         return mid

7.     else if key < A[mid]
8.         high = mid - 1

9.     else
10.        low = mid + 1
11.    end if
12. end while

13. return -1
```

6.6 C Program for Binary Search

```c
#include <stdio.h>

int binarySearch(int array[], int size, int key)
{
    int low = 0;
    int high = size - 1;
    int mid;

    while (low <= high)
    {
        mid = (low + high) / 2;
```

```
12
13          if (array[mid] == key)
14          {
15              return mid;
16          }
17          else if (key < array[mid])
18          {
19              high = mid - 1;
20          }
21          else
22          {
23              low = mid + 1;
24          }
25      }
26
27      return -1;
28  }
29
30  int main()
31  {
32      int array[100];
33      int size, key, result;
34      int i;
35
36      printf("Enter the number of elements: ");
37      scanf("%d", &size);
38
39      printf("Enter %d elements in sorted order:\n", size);
40
41      for (i = 0; i < size; i++)
42      {
43          scanf("%d", &array[i]);
44      }
45
46      printf("Enter the element to search: ");
47      scanf("%d", &key);
48
49      result = binarySearch(array, size, key);
50
51      if (result == -1)
52      {
53          printf("Element not found.\n");
54      }
55      else
56      {
57          printf("Element found at index %d.\n", result);
58          printf("Element found at position %d.\n", result + 1);
59      }
60
61      return 0;
62  }
```

6.7 Sample Input and Output

```
Enter the number of elements: 7
Enter 7 elements in sorted order:
5 10 15 20 25 30 35
Enter the element to search: 25

Element found at index 4.
Element found at position 5.
```

6.8 Complexity of Binary Search

Case	Complexity	Condition
Best Case	$O(1)$	The key is present at the middle index.
Average Case	$O(\log n)$	The search range is repeatedly divided into two halves.
Worst Case	$O(\log n)$	The key is found after the maximum number of divisions or is not present.
Space Complexity	$O(1)$	The iterative implementation requires constant additional memory.

Table 2: Complexity of Binary Search

7 Comparison of Linear Search and Binary Search

Parameter	Linear Search	Binary Search
Array Requirement	The array may be sorted or unsorted.	The array must be sorted.
Method	Checks elements sequentially.	Repeatedly divides the array into two halves.
Technique	Sequential searching.	Divide-and-Conquer.
Best-Case Complexity	$O(1)$	$O(1)$
Worst-Case Complexity	$O(n)$	$O(\log n)$
Suitable For	Small or unsorted arrays.	Large sorted arrays.
Implementation	Simple.	Comparatively more complex.

Table 3: Comparison of Linear Search and Binary Search

8 Solved Lab Question

Question

Consider the following sorted array:

$$A = [5, 12, 18, 23, 37, 45, 56, 67, 72]$$

Write a C program to search for the element 45 using:

(a) Linear Search

(b) Binary Search

Also display the number of comparisons required by each algorithm.

C Program for the Solved Question

```c
#include <stdio.h>

int linearSearch(int array[], int size, int key,
                 int *comparisons)
{
    int i;

    *comparisons = 0;

    for (i = 0; i < size; i++)
    {
        (*comparisons)++;

        if (array[i] == key)
        {
            return i;
        }
    }

    return -1;
}

int binarySearch(int array[], int size, int key,
                 int *comparisons)
{
    int low = 0;
    int high = size - 1;
    int mid;

    *comparisons = 0;

    while (low <= high)
    {
        mid = (low + high) / 2;
```

```
35
36          printf("low = %d, high = %d, mid = %d, "
37                 "A[mid] = %d\n",
38                 low, high, mid, array[mid]);
39
40          (*comparisons)++;
41
42          if (array[mid] == key)
43          {
44              return mid;
45          }
46          else if (key < array[mid])
47          {
48              high = mid - 1;
49          }
50          else
51          {
52              low = mid + 1;
53          }
54      }
55
56      return -1;
57  }
58
59  int main()
60  {
61      int array[] = {5, 12, 18, 23, 37,
62                     45, 56, 67, 72};
63
64      int size = sizeof(array) / sizeof(array[0]);
65      int key = 45;
66
67      int linearResult;
68      int binaryResult;
69      int linearComparisons;
70      int binaryComparisons;
71
72      linearResult = linearSearch(array, size, key,
73                                  &linearComparisons);
74
75      printf("Linear Search Result:\n");
76
77      if (linearResult != -1)
78      {
79          printf("Element %d found at index %d.\n",
80                 key, linearResult);
81          printf("Element %d found at position %d.\n",
82                 key, linearResult + 1);
83      }
84      else
85      {
```

```
86          printf("Element not found.\n");
87      }
88
89      printf("Number of comparisons = %d\n\n",
90             linearComparisons);
91
92      printf("Binary Search Steps:\n");
93
94      binaryResult = binarySearch(array, size, key,
95                                  &binaryComparisons);
96
97      printf("\nBinary Search Result:\n");
98
99      if (binaryResult != -1)
100     {
101         printf("Element %d found at index %d.\n",
102                key, binaryResult);
103         printf("Element %d found at position %d.\n",
104                key, binaryResult + 1);
105     }
106     else
107     {
108         printf("Element not found.\n");
109     }
110
111     printf("Number of comparisons = %d\n",
112            binaryComparisons);
113
114     return 0;
115 }
```

Program Output

```
Linear Search Result:
Element 45 found at index 5.
Element 45 found at position 6.
Number of comparisons = 6

Binary Search Steps:
low = 0, high = 8, mid = 4, A[mid] = 37
low = 5, high = 8, mid = 6, A[mid] = 56
low = 5, high = 5, mid = 5, A[mid] = 45

Binary Search Result:
Element 45 found at index 5.
Element 45 found at position 6.
Number of comparisons = 3
```

Explanation of Linear Search

Linear Search performs the following comparisons:

$$5,\ 12,\ 18,\ 23,\ 37,\ 45$$

The element 45 is found at index 5, which is position 6.
Therefore, the total number of comparisons is:

$$\boxed{6}$$

Explanation of Binary Search

For C array indexing:

$$low = 0, \qquad high = 8$$

Step 1

$$mid = \left\lfloor \frac{0+8}{2} \right\rfloor = 4$$

$$A[4] = 37$$

Since:

$$45 > 37,$$

the right half is selected:

$$low = mid + 1 = 5$$

Step 2

Now:

$$low = 5, \qquad high = 8$$

$$mid = \left\lfloor \frac{5+8}{2} \right\rfloor = \lfloor 6.5 \rfloor = 6$$

$$A[6] = 56$$

Since:

$$45 < 56,$$

the left half is selected:

$$high = mid - 1 = 5$$

Step 3

Now:

$$low = 5, \qquad high = 5$$

$$mid = \left\lfloor \frac{5+5}{2} \right\rfloor = 5$$

$$A[5] = 45$$

Therefore, the element is found at index 5, which is position 6.
The total number of comparisons is:

$$\boxed{3}$$

Final Comparison

Algorithm	Index	Position	Comparisons
Linear Search	5	6	6
Binary Search	5	6	3

Table 4: Results of the Solved Question

For the given sorted array, Binary Search requires fewer comparisons than Linear Search.

9 Question for Students

Consider the following sorted array:

$$A = [3, 8, 14, 21, 29, 35, 42, 50, 63, 78]$$

Write a C program to search for the element 29 using:

1. Linear Search
2. Binary Search

The program must display:

(a) The index and position of the element.

(b) The number of comparisons made by Linear Search.

(c) The values of `low`, `high`, and `mid` during every step of Binary Search.

(d) The number of comparisons made by Binary Search.

(e) A comparison stating which algorithm is more efficient.

Use the following formula in Binary Search:

$$\boxed{mid = \left\lfloor \frac{low + high}{2} \right\rfloor}$$

In the C program, write:

```
mid = (low + high) / 2;
```

Space for Student Observation

10 Result

Linear Search and Binary Search were successfully implemented using C language. Linear Search checks the elements sequentially, whereas Binary Search repeatedly divides a sorted array into two halves.

11 Conclusion

Linear Search is simple and can be applied to both sorted and unsorted arrays. However, its worst-case time complexity is $O(n)$.

Binary Search is more efficient for large sorted arrays because its worst-case time complexity is $O(\log n)$. Therefore, Binary Search is preferred when the data is sorted and frequent searching is required.